Tutorial 4

Member Operators · Arrays of Objects · Valgrind

Review: Operator Overloading

Last week we saw that operators can be overloaded as standalone functions:

```
struct Vec { int x, y; };

Vec operator+(const Vec &v1, const Vec &v2) {
    return { v1.x + v2.x, v1.y + v2.y };
}

ostream &operator<<(ostream &out, const Vec &v) {
    out << v.x << ' ' << v.y;
    return out;   // enables chaining
}
```

- Must return the stream by **reference** to support chaining
- Today: **member** operators, arrays of objects, and Valgrind

This week

1. Member operators and when to use them
2. Operators that must be members
3. Arrays of objects — the default constructor problem
4. Finding memory errors with Valgrind
5. Reading Valgrind reports: leaks and errors

When an operator is declared as a **member function**, the object it is called on plays the role of the **first operand**:

```
struct Vec {
    int x, y;

    Vec operator+(const Vec &other) {   // member: this + other
        return { x + other.x, y + other.y };
    }
    Vec operator*(const int k) {        // this * k
        return { x * k, y * k };
    }
};
```

Expression	Can it be a member?
`v * k`	Yes — `Vec` is the first operand

`k * v`	No — `int` is the first operand; must be a standalone function

1 · k * v — Must Be a Standalone Function

Because `int` is the first operand, `k * v` cannot be a member of `Vec`. Write it as an external function and reuse the member:

```
Vec operator*(const int k, const Vec &v) {
    return v * k;   // reuse the member operator*(int)
}
```

Advice: if you overload an arithmetic operator, overload the assignment version too, and implement the former in terms of the latter:

```
Vec &operator+=(Vec &v1, const Vec &v2) {
    v1.x += v2.x;
    v1.y += v2.y;
    return v1;
}

Vec operator+(const Vec &v1, const Vec &v2) {
```

```
    Vec tmp{v1};
    return tmp += v2;   // implement + in terms of +=
}
```

1 · I/O Operators Must Be Standalone

Writing `operator<<` as a member makes `Vec` the first operand — forcing the awkward call `v << cout`:

```
struct Vec {
    // Wrong: call would be  v << cout
    ostream &operator<<(ostream &out) {
        return out << x << ' ' << y;
    }
};
```

Fix: define `operator<<` and `operator>>` as **standalone** functions:

```
ostream &operator<<(ostream &out, const Vec &v) {
    return out << v.x << ' ' << v.y;
}
// now:  cout << v  ← natural
```

- The stream is the **first** argument — cannot be a member of `Vec`

- Return the stream by reference to support chaining: `cout << a << b`

Some operators can **only** be defined as member functions:

Operator	Purpose
`operator=`	copy/move assignment: `a = b`
`operator[]`	subscripting: `a[i]`
`operator->`	member access through a pointer-like object: `p->m`
`operator()`	function-call syntax: `f(args)`
`operator T`	conversion of the object to type `T`

These must be members because the language requires the left-hand side to be an object of the class. Defining them as free functions is a compile error.

Array creation calls the **default constructor** on each element. If no default constructor exists, the array cannot be created:

```
struct Vec {
    int x, y;
    Vec(int x, int y) : x{x}, y{y} {}  // no default constructor!
};

Vec *vp = new Vec[10];     // Error: no default constructor
Vec moreVectors[15];       // Error: no default constructor
```

Options:

1. Provide a default constructor — only if it makes semantic sense
2. For stack arrays: supply an initializer for each element
3. For heap arrays: create an **array of pointers**

Option 2 — stack array with per-element initializers:

```
Vec moreVectors[3] = { Vec{0,0}, Vec{1,3}, Vec{2,4} };
```

Option 3 — heap array of pointers (our running Valgrind example):

```
Vec **vp = new Vec*[5];          // array of pointers

vp[0] = new Vec{0, 0};
vp[1] = new Vec{1, 3};
// ...

for (int i = 0; i < 5; ++i) delete vp[i];  // 1. free each Vec
delete [] vp;                                // 2. free the array
```

- Delete order matters: free the individual objects **first**, then the pointer array
- Forgetting either step is a memory leak that Valgrind will catch

Valgrind runs your program under a watchful eye — it tracks every heap allocation and memory access. Its default tool **Memcheck** detects:

- **Memory leaks** — a `new` with no matching `delete`
- **Invalid reads/writes** — e.g. going past the end of an array
- **Use after free** — touching memory that was already deleted
- **Invalid or mismatched frees** — deleting twice, or `delete` where `delete []` was needed
- **Use of uninitialized values**

These bugs are hard to catch by running the program normally — it can seem to work perfectly and then crash when used slightly differently. Valgrind finds them, although it only tells you the **problem**, not the **solution**.

Compile with `-g` so reports include file names and line numbers, then put `valgrind` in front of your command:

```
g++20 -g vecs.cc -o vecs
valgrind ./vecs
valgrind --leak-check=full ./vecs   // show where leaked blocks were
allocated
```

A **clean** run ends like this:

```
==1563== HEAP SUMMARY:
==1563==   in use at exit: 0 bytes in 0 blocks
==1563==   total heap usage: 8 allocs, 8 frees, 75,072 bytes allocated
==1563==
==1563== All heap blocks were freed -- no leaks are possible
==1563==
==1563== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Three things to check: allocs = frees · in use at exit: 0 · ERROR SUMMARY: 0 errors

If blocks remain on the heap, the **LEAK SUMMARY** sorts them into categories:

Category	Meaning
`definitely lost`	No pointer to the block exists. Always a bug — fix these first.
`indirectly lost`	Only reachable through other lost blocks (e.g. the rest of a linked list whose head was lost). Fixing the `definitely lost` root fixes these too.
`possibly lost`	Only a pointer into the **middle** of the block exists.
`still reachable`	A pointer exists at exit, but the block was never freed.

Run with `--leak-check=full` to get a stack trace for each leak showing **where the block was allocated**. Without it you only learn the byte count, which is not enough to act on.

```
==1700== Invalid write of size 4
==1700==    at 0x1091F2: main (vecs.cc:14)
==1700==  Address 0x4d51048 is 0 bytes inside a block of size 8 free'd
==1700==    at 0x483708B: operator delete(void*, unsigned long) (...)
==1700==    by 0x1091C5: main (vecs.cc:12)
==1700==  Block was alloc'd at
==1700==    at 0x4835DEF: operator new(unsigned long) (...)
==1700==    by 0x109186: main (vecs.cc:9)
```

Part	**Meaning**
First line	What went wrong and how many bytes — `"Invalid write of size 4"`
Stack trace	Where it happened — look for the first line that is **your** code
`"inside a block … free'd"`	Use after free — you touched memory already deleted

`"bytes after a block … alloc'd"`	Out of bounds — you ran off the end of an array

Recall the heap array of pointers from the previous section. This version is **correct**, and Valgrind reports no errors and no leaks for it:

```
struct Vec {
    int x, y;
    Vec(int x, int y) : x{x}, y{y} {}
};

int main() {
    Vec **vp = new Vec*[5];                          // 1 block: 5 pointers = 40
bytes
    for (int i = 0; i < 5; ++i) vp[i] = new Vec{i, i};  // 5 blocks of 8
bytes
    for (int i = 0; i < 5; ++i) delete vp[i];    // free each Vec ...
    delete [] vp;                                 // ... then the array itself
}
```

Now we introduce one mistake at a time. In `vecs.cc` each one is selected with a `-D` flag at compile time:

```
g++20 -g -DMISTAKE=1 vecs.cc -o vecs
valgrind --leak-check=full ./vecs
```

Without the `delete vp[i]` loop, `delete [] vp` frees only the array of pointers. The five `Vec`s are now unreachable:

```
// for (int i = 0; i < 5; ++i) delete vp[i];   <-- missing
delete [] vp;
```

Valgrind reports:

```
==1701== 40 bytes in 5 blocks are definitely lost in loss record 1 of 1
==1701==    at 0x4835DEF: operator new(unsigned long) (...)
==1701==    by 0x109216: main (vecs.cc:9)
```

The trace points at `new Vec{i, i}` inside the loop, which is where the leaked blocks were allocated. It does **not** point at the place where the missing `delete` belongs — you have to work that out.

(g++ already warns about this one at compile time.)

```
delete vp;   // vp came from new Vec*[5], so this should be delete [] vp
```

Valgrind reports:

```
==1702== Mismatched free() / delete / delete []
==1702==    at 0x483708B: operator delete(void*, unsigned long) (...)
==1702==    by 0x109254: main (vecs.cc:11)
==1702==  Address 0x4d51040 is 0 bytes inside a block of size 40 alloc'd
```

Always match `new[]` with `delete []`. Using plain `delete` on an array is undefined behaviour.

The array has indices 0 to 4, but the loop runs to 5:

```
for (int i = 0; i <= 5; ++i) vp[i] = new Vec{i, i};  // should be i < 5
```

Valgrind reports:

```
==1703== Invalid write of size 8
==1703==    at 0x1091E1: main (vecs.cc:9)
==1703==  Address 0x4d51068 is 0 bytes after a block of size 40 alloc'd
```

The extra `Vec` created by the last iteration is never deleted, so Valgrind also reports it as lost. The phrase **“0 bytes after a block of size 40 alloc’d”** tells you exactly that you ran one step past the end of the array.

After `delete vp[0]`, the pointer `vp[0]` still holds the old address (a **dangling pointer**). Using it, or deleting it again, is an error:

```
for (int i = 0; i < 5; ++i) delete vp[i];
vp[0]->x = 9;   // use after free
delete vp[0];   // double delete
delete [] vp;
```

Valgrind reports:

```
==1704== Invalid write of size 4
==1704==    at 0x1092A3: main (vecs.cc:12)
==1704==  Address 0x4d51090 is 0 bytes inside a block of size 8 free'd
...
==1704== Invalid free() / delete / delete [] / realloc()
==1704==    at 0x483708B: operator delete(void*, unsigned long) (...)
```

```
==1704==    by 0x1092B5: main (vecs.cc:13)
==1704==  Address 0x4d51090 is 0 bytes inside a block of size 8 free'd
```

A good habit is to set a pointer to `nullptr` right after deleting it:

```
delete vp[i];
vp[i] = nullptr;
```

Deleting `nullptr` is harmless, and **using** it fails immediately instead of silently corrupting memory. Try `MISTAKE=5` to see Valgrind report **no errors**.

Mistake	**What Valgrind reports**
Elements never deleted	`definitely lost`
`delete` for an array	`Mismatched free() / delete / delete []`
Index out of range	`Invalid read/write` — “bytes after a block … alloc’d”
Use after delete	`Invalid read/write` — “bytes inside a block … free’d”
Double delete	`Invalid free() / delete / delete [] / realloc()`

Common Pitfalls — Quick Reference

Mistake	Fix
Writing `operator<<` as a member	Make it a standalone function: `(ostream&, const T&)`
Using `operator+` without implementing `+=`	Implement `+=` first; write `+` in terms of it
Array of objects with no default constructor	Add one, use per-element initializers, or use an array of pointers
Using `delete` instead of `delete []`	Always match `new[]` with `delete []`
Not running `--leak-check=full`	Use the flag to get allocation stack traces for leaked memory
Dangling pointer after `delete`	Set the pointer to `nullptr` immediately after deleting